

Spring Boot Data REST, Many to many relationship

Many to many relationship in Spring Boot Data REST

Suppose we have a mechanic that works in a workshop. A garage may have several mechanics. A mechanic can repair many cars. A car can be repaired by several mechanics. We need to model this with Spring Data REST.

We can use three entity classes: Mechanic, Garage, and Car. The Mechanic entity would have a many-to-many relationship with the Car entity, and the Garage entity would have a one-to-many relationship with the Mechanic entity.

You can annotate the Mechanic entity with `@ManyToMany` and use the `@JoinTable` annotation to specify the join table that will be used to link Mechanics and Cars. The Car entity would have a corresponding `@ManyToMany` annotation to establish the relationship in the other direction.

The Garage entity would have a `@OneToMany` relationship with Mechanic, and you can use the `@JoinColumn` annotation to specify the foreign key that links Mechanic to Garage.

Spring Data REST will then automatically expose REST endpoints for these entities, allowing you to create, read, update, and delete records using HTTP requests.

We can specify the join table between Mechanic and Car in Spring Data REST, and also include an additional attribute, such as the date of repair, specific to each Mechanic-Car relationship. To specify the join table and include an additional attribute, you can create a separate entity class that represents the join table. This class should have two `@ManyToOne` annotations to represent the Mechanic and Car entities, and an additional attribute for the date of repair.

Example with jointable and a ManyToMany relationship

```
@Entity
public class MechanicCarRepair {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "mechanic_id")
    private Mechanic mechanic;
```

```

    @ManyToOne
    @JoinColumn(name = "car_id")
    private Car car;

    private LocalDate dateOfRepair;

    // constructors, getters, and setters
}

```

The Mechanic and Car entities have a many-to-many relationship through the MechanicCarRepair entity, which represents the join table. The @ManyToOne annotations specify the relationship between MechanicCarRepair and Mechanic/Car entities, and the @JoinColumn annotations specify the foreign keys that link the entities.

You can then use Spring Data REST to expose endpoints for MechanicCarRepair, Mechanic, and Car entities, allowing you to create, read, update, and delete records using HTTP requests.

Here's the code for what Mechanic and Car entities could look like:

```

@Entity
public class Mechanic {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany(mappedBy = "mechanics")
    private List<Car> cars = new ArrayList<>();

    // constructors, getters, and setters
}

@Entity
public class Car {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String make;

    private String model;

    @ManyToMany
    @JoinTable(
        name = "mechanic_car",
        joinColumns = @JoinColumn(name = "car_id"),

```

```

        inverseJoinColumns = @JoinColumn(name = "mechanic_id"))
    private List<Mechanic> mechanics = new ArrayList<>();

    // constructors, getters, and setters
}

```

The Mechanic entity has an ID, a name, and a many-to-many relationship with Car through a list of cars. The @ManyToMany annotation specifies that the relationship is bidirectional, and the "mappedBy" attribute specifies that the owning side of the relationship is the Car entity.

The Car entity has an ID, a make, a model, and a many-to-many relationship with Mechanic through a list of mechanics. The @ManyToMany annotation specifies the relationship and the @JoinTable annotation specifies the name of the join table and the foreign keys used to link Mechanic and Car.

If you are using a join table to model the many-to-many relationship between Car and Mechanic, then you only need to specify the join table in one of the entities. It doesn't matter which one you choose, as long as you are consistent in your choice across your entire application.

In the example I provided earlier where we used a join table, we specified the @JoinTable annotation on the Car entity. This is because we assumed that the relationship is bidirectional and the owning side is the Car entity. Therefore, the Mechanic entity only needed to specify the other side of the relationship using the "mappedBy" attribute in the @ManyToMany annotation.

In this example, the @JoinTable annotation is only specified on the Car entity, and the Mechanic entity uses the "mappedBy" attribute to specify the other side of the relationship.

If you're using a separate join entity, then you don't need to specify the join table in either the Car or Mechanic entities, because the relationship is modeled through the join entity instead.

Without specifying a Jointable and only ManyToOne, OneToMany relationship

It is not necessary to specify the join table in the Mechanic entity when using a separate join entity like MechanicCarRepair to model the relationship between Mechanic and Car. In this case, the relationship between Mechanic and MechanicCarRepair would be a one-to-many relationship, and the Mechanic entity would have a collection of MechanicCarRepair instances.

Here's an example of what the Mechanic and Car entity classes could look like in Java using Spring Data JPA annotations, with the Mechanic-Car relationship modeled using MechanicCarRepair as a separate join entity:

```

@Entity
public class Mechanic {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "mechanic")

```

```

        private List<MechanicCarRepair> repairs = new ArrayList<>();

        // constructors, getters, and setters
    }

@Entity
public class Car {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String make;

    private String model;

    @OneToMany(mappedBy = "car")
    private List<MechanicCarRepair> repairs = new ArrayList<>();

    // constructors, getters, and setters
}

@Entity
public class MechanicCarRepair {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "mechanic_id")
    private Mechanic mechanic;

    @ManyToOne
    @JoinColumn(name = "car_id")
    private Car car;

    private LocalDate dateOfRepair;

    // constructors, getters, and setters
}

```

Here the Mechanic entity has an ID, a name, and a one-to-many relationship with MechanicCarRepair through a list of repairs. The @OneToMany annotation specifies that the relationship is bidirectional, and the "mappedBy" attribute specifies that the owning side of the relationship is the MechanicCarRepair entity.

The Car entity has an ID, a make, a model, and a one-to-many relationship with MechanicCarRepair through a list of repairs. The @OneToMany annotation specifies the relationship and the

"mappedBy" attribute specifies that the owning side of the relationship is the MechanicCarRepair entity.

The MechanicCarRepair entity has an ID, a many-to-one relationship with Mechanic through a mechanic foreign key, a many-to-one relationship with Car through a car foreign key, and an additional attribute for the date of repair.

You can then use Spring Data JPA to persist and query these entities in a database.

ManyToMany vs several OneToMany, ManyToOne relationships

The choice between using a many-to-many relationship with a join table versus using two one-to-many relationships with a separate join entity depends on your specific use case and design preferences.

Here are some factors to consider when deciding between the two options:

Using a many-to-many relationship with a join table:

- This option is simpler and requires fewer entities to model the relationship.
- However, it may not be as flexible as using a separate join entity, as you may need to add additional attributes to the join table in the future.

Using two one-to-many relationships with a separate join entity:

- This option provides more flexibility, as you can add additional attributes to the join entity if needed.
- However, it requires an additional entity to model the relationship, which can make the code more complex.

In general, if the relationship between Car and Mechanic is simple and doesn't require any additional attributes beyond the IDs of the related entities, using a many-to-many relationship with a join table is a good option. It is simpler and requires fewer entities to model the relationship.

If you anticipate that you may need to add additional attributes to the relationship in the future, or if the relationship is more complex (e.g., a car can have multiple mechanics, but each mechanic has a different role in the repair process), then using two one-to-many relationships with a separate join entity may be a better option.

Ultimately, the best approach depends on the specific requirements and design of your application.

Example for the case of one Join entity with several OneToMany relationships

In this example we will use a service too, but you can make service actions directly in the controller if they are not very complicated and not reusable.

```
@RestController
@RequestMapping("/mechanic-car-repairs")
public class MechanicCarRepairController {
```

```

private final MechanicCarRepairRepository repository;
private final CarRepository carRepository;
private final MechanicRepository mechanicRepository;

public MechanicCarRepairController(
    MechanicCarRepairRepository repository,
    CarRepository carRepository,
    MechanicRepository mechanicRepository) {
    this.repository = repository;
    this.carRepository = carRepository;
    this.mechanicRepository = mechanicRepository;
}

@GetMapping
public List<MechanicCarRepair> getAll() {
    return repository.findAll();
}

@GetMapping("/{id}")
public MechanicCarRepair getById(@PathVariable Long id) {
    return repository.findById(id)
        .orElseThrow(() ->
            new ResponseStatusException(HttpStatus.NOT_FOUND));
}

@PostMapping
public MechanicCarRepair create(
    @RequestBody MechanicCarRepairRequest request) {
    Car car = carRepository.findById(request.getCarId())
        .orElseThrow(() ->
            new ResponseStatusException(HttpStatus.NOT_FOUND,
                "Car not found"));

    Mechanic mechanic = mechanicRepository
        .findById(request.getMechanicId())
        .orElseThrow(() ->
            new ResponseStatusException(HttpStatus.NOT_FOUND,
                "Mechanic not found"));

    MechanicCarRepair repair =
        new MechanicCarRepair(request.getDate(), car, mechanic);

    return repository.save(repair);
}
}

```

```

public interface MechanicCarRepairRepository extends
    JpaRepository<MechanicCarRepair, Long> {
}

@Service
public class MechanicCarRepairService {
    private final MechanicCarRepairRepository repository;

    public MechanicCarRepairService(
        MechanicCarRepairRepository repository) {

        this.repository = repository;
    }

    public List<MechanicCarRepair> getAll() {
        return repository.findAll();
    }

    public MechanicCarRepair getById(Long id) {
        return repository.findById(id)
            .orElseThrow(() ->
                new ResponseStatusException(HttpStatus.NOT_FOUND));
    }

    public MechanicCarRepair create(MechanicCarRepairRequest request) {
        Car car = carRepository.findById(request.getCarId())
            .orElseThrow(() ->
                new ResponseStatusException(HttpStatus.NOT_FOUND,
                    "Car not found"));

        Mechanic mechanic =
            mechanicRepository.findById(request.getMechanicId())
                .orElseThrow(() ->
                    new ResponseStatusException(HttpStatus.NOT_FOUND,
                        "Mechanic not found"));

        MechanicCarRepair repair =
            new MechanicCarRepair(request.getDate(), car, mechanic);
        return repository.save(repair);
    }
}

public class MechanicCarRepair {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
}

```

```

    private LocalDate date;

    @ManyToOne
    @JoinColumn(name = "car_id")
    private Car car;

    @ManyToOne
    @JoinColumn(name = "mechanic_id")
    private Mechanic mechanic;

    // constructors, getters, and setters
}

public class MechanicCarRepairRequest {
    private LocalDate date;
    private Long carId;
    private Long mechanicId;

    // constructors, getters, and setters
}

```

In this example, the MechanicCarRepair entity has two @ManyToOne relationships with Car and Mechanic, respectively. The MechanicCarRepairRepository is a simple Spring Data JPA repository that extends the JpaRepository interface. The MechanicCarRepairService class provides methods for retrieving and creating MechanicCarRepair instances using the repository and the CarRepository and MechanicRepository instances.

The MechanicCarRepairController class is a Spring MVC REST controller that maps HTTP requests to the corresponding MechanicCarRepairService methods. In the create method, the MechanicCarRepairRequest parameter is used to create a new MechanicCarRepair instance, which is then saved to the database using the MechanicCarRepairRepository.

The code for the Car and Mechanic entities as well:

```

public class Car {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String make;

    private String model;

    @OneToMany(mappedBy = "car", cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<MechanicCarRepair> repairs = new ArrayList<>();
}

```

```

        // constructors, getters, and setters
    }

    public class Mechanic {
        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;

        private String name;

        @OneToMany(mappedBy = "mechanic", cascade = CascadeType.ALL,
            orphanRemoval = true)
        private List<MechanicCarRepair> repairs = new ArrayList<>();

        // constructors, getters, and setters
    }

```

In these examples, Car and Mechanic have @OneToMany relationships with MechanicCarRepair. The repairs field in each entity specifies the inverse side of the relationship, and the mappedBy attribute is used to indicate that the relationship is managed by the car and mechanic fields in MechanicCarRepair, respectively.

Note that in this example, the cascade attribute is set to CascadeType.ALL, which means that any changes made to Car, Mechanic, or MechanicCarRepair instances will be cascaded to the other entities in the relationship. The orphanRemoval attribute is set to true, which means that any MechanicCarRepair instances that are no longer associated with a Car or Mechanic instance will be deleted from the database.

MySQL tables and fake data

```

CREATE TABLE car (
    id BIGINT NOT NULL AUTO_INCREMENT,
    make VARCHAR(255) NOT NULL,
    model VARCHAR(255) NOT NULL,
    PRIMARY KEY (id)
);

CREATE TABLE mechanic (
    id BIGINT NOT NULL AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    PRIMARY KEY (id)
);

CREATE TABLE mechanic_car_repair (
    id BIGINT NOT NULL AUTO_INCREMENT,

```

```

car_id BIGINT NOT NULL,
mechanic_id BIGINT NOT NULL,
date DATE NOT NULL,
PRIMARY KEY (id),
FOREIGN KEY (car_id) REFERENCES car (id),
FOREIGN KEY (mechanic_id) REFERENCES mechanic (id)
);

```

```

INSERT INTO car (make, model) VALUES
('Toyota', 'Corolla'),
('Honda', 'Civic'),
('Ford', 'F-150');

```

```

INSERT INTO mechanic (name) VALUES
('John Smith'),
('Sarah Lee'),
('Mike Johnson');

```

```

INSERT INTO mechanic_car_repair (car_id, mechanic_id, date) VALUES
(1, 2, '2022-01-15'),
(1, 3, '2022-01-17'),
(2, 1, '2022-01-18'),
(3, 2, '2022-01-20'),
(3, 3, '2022-01-21');

```

Temporaly disble Security

While developing, perhaps, we want to disable security checks. There are two main ways:

1. Create a new configuration that disables security. You can create a new configuration class that extends `WebSecurityConfigurerAdapter` and overrides the `configure()` method to disable security. Here is an example:

```

@Configuration
@Order(SecurityProperties.BASIC_AUTH_ORDER)
public class SecurityConfiguration extends
    WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests()
            .anyRequest().permitAll()
            .and()
            .csrf().disable();
    }
}

```

We need this dependency:

```
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-config</artifactId>
</dependency>
```

This configuration disables security for all endpoints by permitting access to all requests. Additionally, it disables Cross-Site Request Forgery (CSRF) protection, which is not recommended in production environments but may be useful for testing and development.

2. Use Spring Boot's built-in security properties to disable security. You can add the following properties to your application.properties or application.yml file to disable security:

```
spring.security.user.password=none
spring.security.user.name=none
```

This configuration sets the username and password to "none", effectively disabling authentication for all requests.

Note that this method is not recommended for production environments, as it leaves your application completely open to attack. However, it can be useful for testing and development.

First way is now deprecated, while looking for a new solution both still correct.